

Magnitude-Free Entropy Extractor (MFE-31)

A minimalist linear extractor for jitter-based entropy sources

shahiN Noursalehi
(mixoftix@gmail.com)

Introduction

The Magnitude-Free Entropy Extractor (MFE-31) is a lightweight, deterministic entropy-extraction primitive designed for constrained microcontrollers (e.g., AVR). It transforms a raw 32-bit jitter-derived sample into a uniform 31-bit entropy block by neutralizing magnitude dominance in the most significant bit (MSB). The extractor is computationally minimal, requiring only bit-rotation, a single subtraction, and a normalization step.

MFE-31 is intended for environments where hashing-based conditioners (SHA-256, AES-CTR) are infeasible due to memory or cycle constraints, yet a mathematically sound entropy-conditioning step is required.

1. Raw Entropy Model

Let $x \in \{0, 1\}^{32}$ be a raw 32-bit output from a jitter-based entropy source. We assume:

- X contains at least 31 bits of min-entropy.
- The MSB of X may exhibit magnitude dominance or structural bias.
- Lower and middle bits of X behave as high-entropy jitter samples.

This matches empirical behavior of ring-oscillator and jitter TRNGs.

2. Magnitude Dominance Problem

The MSB of a 32-bit integer contributes a fixed magnitude of 2^{31} . If the MSB is biased or structurally correlated, the numeric distribution of X becomes skewed.

Magnitude dominance is defined as:

$$\text{MagDom}(X) = X_{31} \cdot 2^{31}$$

where X_{31} is the MSB.

To obtain a uniform entropy block, this magnitude by MSB must be neutralized.

Binary (32-bit)	Decimal Value
10000000000000000000000000000000	2147483648
10010000000000000000000000000000	2415919104
10100000000000000000000000000000	2684354560
10110000000000000000000000000000	2952790016
11000000000000000000000000000000	3221225472
11010000000000000000000000000000	3489660928
11100000000000000000000000000000	3758096384
11110000000000000000000000000000	4026531840
11111000000000000000000000000000	4160749568
11111100000000000000000000000000	4227858432
11111110000000000000000000000000	4261412864
11111111000000000000000000000000	4278190080
11111111100000000000000000000000	4286578688
11111111110000000000000000000000	4290772992
11111111111000000000000000000000	4292870144

Binary (32-bit)	Decimal Value
01000000000000000000000000000000	1073741824
01010000000000000000000000000000	1342177280
01100000000000000000000000000000	1610612736
01110000000000000000000000000000	1879048192
01111000000000000000000000000000	2013265920
01111100000000000000000000000000	2080374784
01111110000000000000000000000000	2113929216
01111111000000000000000000000000	2130706432
01111111100000000000000000000000	2139095040
01111111110000000000000000000000	2143289344
01111111111000000000000000000000	2145386496
01111111111100000000000000000000	2146435072
01111111111110000000000000000000	2146959360
01111111111111000000000000000000	2147221504
01111111111111100000000000000000	2147352576

3. Bit-Position Normalization (Circular Scan)

We define a rotation operator:

$$\text{ROTL32}(X, 1)$$

which performs a 32-bit circular left shift.

We perform a deterministic scan:

$$\text{While MSB}(X) = 0:$$

$$X \leftarrow \text{ROTL32}(X, 1)$$

This procedure rotates the bitstring until a high-entropy bit reaches the MSB position. Because rotation is a permutation of the 32-bit space, no entropy is lost during scanning.

This step ensures:

$$X_{31}=1$$

and guarantees that the MSB now contains a genuine entropy bit.

5. Magnitude Neutralization

After normalization, the integer value of x lies in:

$$2^{31} \leq x \leq 2^{32}-1$$

To remove the fixed magnitude contribution of the MSB, we subtract 2^{31} :

$$L = x - 2^{31}$$

This produces:

$$L \in [0, 2^{31}-1]$$

The subtraction is equivalent to masking the MSB:

$$L = x \& 0x7FFFFFFF$$

but avoids introducing a new magnitude discontinuity.

This step is a **linear entropy extractor**:

$$\text{Extract}(X) = X \ll 2^{31}$$

It compresses the raw 32-bit block into a uniform 31-bit entropy block.

6. Output Distribution

The extracted block L is uniformly distributed over:

$$\{0, 1, \dots, 2^{31}-1\}$$

To obtain a floating-point value in $[0, 1)$, we compute:

$$U = L / 2^{31}$$

This normalization preserves uniformity.

7. Security Properties

Deterministic	The extractor is fully deterministic and stateless.
Entropy-Preserving	The extractor retains 31 bits of entropy and discards only the biased MSB.
Bias-Neutralizing	Magnitude dominance is eliminated by: 1. rotating until MSB contains entropy 2. subtracting the fixed magnitude contribution
Permutation-Safe	Rotation is a bijection on $\{0,1\}^{32}$, ensuring no entropy loss during scanning.
Minimal Computational Cost	Requires: • ≤ 31 rotations • one subtraction • one division

Suitable for 8-bit microcontrollers.

8. Comparison to Standard Extractors

Extractor Type	Mechanism	Cost	Effect on Magnitude / Bias	Notes
Von Neumann	Pairwise debiasing	Low	Removes <i>positional</i> bias only	Does not address magnitude dominance
Linear Extractor	XOR / subtraction	Low	XOR preserves MSB dominance ; subtraction removes fixed offsets	XOR is linear over GF(2), subtraction is linear over integers
Hash Extractor	SHA-256 / SHA-512	High	Strong mixing; removes all structural bias	Expensive for microcontrollers
AES-CTR Conditioner	Block cipher whitening	High	Strong conditioning; removes magnitude and positional bias	Used in Intel/AMD TRNGs
<u>MFE-31</u>	Rotation + subtraction	Very Low	Removes MSB magnitude dominance ; compresses to uniform 31-bit block	Ideal for jitter TRNGs

MFE-31 is a **minimalist linear extractor**, optimized for constrained environments.

9. Formal Definition

Let:

- $X \in \{0, 1\}^{32}$ be raw entropy
- $\text{ROTL32}(X, 1)$ be circular left shift
- $\text{MSB}(X)$ be the most significant bit

Then:

Magnitude-Free Entropy Extractor

$\text{MFE}(X) =$	$\text{MFE}(\text{ROTL32}(X, 1))$	if $\text{MSB}(X) = 0$
	$X - 2^{31}$	if $\text{MSB}(X) = 1$

Output:

$$L = \text{MFE}(X)$$

Uniform float:

$$U = L / 2^{31}$$

Conclusion

The Magnitude-Free Entropy Extractor (MFE-31) is a compact, mathematically sound entropy-conditioning primitive suitable for jitter-based TRNGs on constrained microcontrollers. It neutralizes magnitude dominance, preserves entropy, and produces uniform output with minimal computational overhead.

Acknowledgement

This article was developed with the assistance of **Microsoft Copilot**, whose explanations, structure, and clarity helped refine the concepts and make them accessible for practitioners.

Appendix I – Magnitude Collapse Table

This table shows how right-shifting a 32-bit upper-half value collapses magnitude by powers of two, demonstrating the dominance of the MSB:

Shift	MIN value	MAX value	Range width
0	2147483648	4294967295	2147483647
1	1073741824	2147483647	1073741823
2	536870912	1073741823	536870911
3	268435456	536870911	268435455
4	134217728	268435455	134217727
5	67108864	134217727	67108863
6	33554432	67108863	33554431
7	16777216	33554431	16777215
8	8388608	16777215	8388607
9	4194304	8388607	4194303
10	2097152	4194303	2097151

Appendix II – Runtime Sample

Below is a real execution trace by actual device output demonstrating the Magnitude-Free rotation and entropy extraction:

```
Begin Random_Weaky
Begin Weaky_Factor
  ____TIMER: 76750
  ____COUNTER: 48
  ____WEAKY: 88347691
End Weaky_Factor

[_Magnitude Freed] 000010101000100000010100001010110 <- 176695382
[_Magnitude Freed] 000101010001000000101000010101100 <- 353390764
[_Magnitude Freed] 001010100010000001010000101011000 <- 706781528
[_Magnitude Freed] 010101000100000010100001010110000 <- 1413563056
[_Magnitude Freed] 101010001000000101000010101100000 <- 2827126112
-----
Weaky_ENTROPY: 679642464
Weaky_RANDOM: 0.316483197
-----
End Random_Weaky
```

Appendix III – Data Flow in (MFE-31)

